

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH

ID: BCS2024542

ROLL NO: 2410303257

Lab Assignment 14: Sorted Check, Search and Fibonacci

Experiment No. 14.1 - Recursive sorted check

Experiment Title:

Recursive sorted check

Aim:

To check whether an array is in ascending order using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base case: if i >= a.length - 1 return true.
2. If a[i] > a[i + 1] return false.
3. Otherwise return isSorted(a, i + 1).
4. Call it from main with i = 0 and print the result.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 14, Task 1
// Question: Write the method boolean isSorted(int[] a, int i) that
// returns true when the values from index i to the end are in
// ascending order. Call it on {2, 4, 6, 8, 10} with i = 0 and print
// the result.

public class Main {
    static boolean isSorted(int[] a, int i) {
        if (i >= a.length - 1) {
            return true;
        }
        if (a[i] > a[i + 1]) {
            return false;
        }
        return isSorted(a, i + 1);
```

```
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8, 10};
        System.out.println(isSorted(a, 0));
    }
}
```

Input:

Array {2, 4, 6, 8, 10}, i = 0.

Output:

```
true
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The array is in ascending order, so the method returned true.

Viva-Voce Questions:

- Why is the base case a.length - 1?
- When does the method return false?
- What does the method return for a one-element array?

Experiment No. 14.2 - Recursive search by index

Experiment Title:

Recursive search by index

Aim:

To find the index of a key from index i onwards using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base case: if i == a.length return -1.
2. If a[i] == key return i.
3. Otherwise return findIndex(a, key, i + 1).
4. Call it from main with key 80 and i = 0.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 14, Task 2
// Question: Write the method int findIndex(int[] a, int key, int i)
// that returns the index of key from index i onwards, or -1 if
// absent. Call it on {45, 67, 80, 52, 90}, key 80, i = 0, and print
// it.

public class Main {
    static int findIndex(int[] a, int key, int i) {
        if (i == a.length) {
            return -1;
        }
        if (a[i] == key) {
            return i;
        }
        return findIndex(a, key, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(findIndex(a, 80, 0));
    }
}
```

Input:

Array {45, 67, 80, 52, 90}, key = 80, i = 0.

Output:

```
2
```

Time / Space Complexity: Time O(n); Space O(n).

Result:

The program was compiled and executed successfully. The key 80 was found at index 2 using recursion.

Viva-Voce Questions:

- How is this similar to linear search?
- What is returned when the key is absent?
- Why does recursion use more space than a loop?

Experiment No. 14.3 - Recursive Fibonacci

Experiment Title:

Recursive Fibonacci

Aim:

To compute the nth Fibonacci number using recursion.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Recursion can check a property of an array by examining one position and delegating the rest to the next call. A recursive search returns the index when a[i] matches the key and -1 when the index passes the end. The Fibonacci sequence is defined as fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); it needs two base cases and makes two recursive calls, so the plain recursive version takes exponential time.

Algorithm / Procedure:

1. Base cases: if n == 0 return 0; if n == 1 return 1.
2. Recursive case: return fib(n - 1) + fib(n - 2).
3. Call fib(7) from main and print the result.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 14, Task 3
// Question: Write the method int fib(int n) using recursion, where
// fib(0) is 0, fib(1) is 1, and otherwise fib(n) is fib(n - 1) plus
// fib(n - 2). Print fib(7).

public class Main {
    static int fib(int n) {
        if (n == 0) {
            return 0;
        }
        if (n == 1) {
            return 1;
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fib(7));
    }
}
```

Input:

n = 7.

Output:

```
13
```

Time / Space Complexity: Time $O(2^n)$; Space O(n).

Result:

The program was compiled and executed successfully. fib(7) was computed recursively and equals 13.

Viva-Voce Questions:

- Why does fib need two base cases?
- What is the time complexity of recursive Fibonacci?
- Why is the same value computed many times?